

```
!pip install gensim
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart
```

```
import gensim
import gensim.downloader as api
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

```
print("Loading model... this may take a minute...")
wv = api.load('word2vec-google-news-300')
print(f"Vocabulary size: {len(wv.index_to_key)}")
word = 'computer'
vector = wv[word]
print(f"\nNumerical vector for '{word}' (first 10 dimensions):\n", vector[:10])
print(f"\nTotal dimensions: {len(vector)}")
```

```
Loading model... this may take a minute...
[=====] 100.0% 1662.8/1662.8MB downloaded
Vocabulary size: 3000000
```

```
Numerical vector for 'computer' (first 10 dimensions):
[ 0.10742188 -0.20117188  0.12304688  0.21191406 -0.09130859  0.21679688
 -0.13183594  0.08300781  0.20214844  0.04785156]
```

```
Total dimensions: 300
```

```
import gensim.downloader as api

# Load the GloVe pre-trained model (100-dimensional vectors)
print("Loading GloVe model... This may take a moment.")
glove_wv = api.load('glove-wiki-gigaword-100')

# Find the numerical vector representation
word = 'intelligence'
if word in glove_wv:
    vector = glove_wv[word]
    print(f"Numerical vector for '{word}' (first 10 dimensions):\n", vector[:10])
    print(f"\nTotal dimensions: {len(vector)}")
else:
    print(f"Word '{word}' not found in GloVe vocabulary.")
```

```
Loading GloVe model... This may take a moment.
[=====] 100.0% 128.1/128.1MB downloaded
Numerical vector for 'intelligence' (first 10 dimensions):
[-0.31101 -0.43291  0.77734 -0.31115  0.052934 -0.8502 -0.35372
 -0.70531  0.084464  0.88768 ]
```

Total dimensions: 100

```
import gensim.downloader as api

# Load pre-trained Word2Vec model
print("Loading Word2Vec model (Google News 300)... This may take a minute.")
wv = api.load('word2vec-google-news-300')

# 1. Check similarity between two words
word1 = 'king'
word2 = 'queen'
similarity = wv.similarity(word1, word2)
print(f"Similarity between '{word1}' and '{word2}': {similarity:.4f}")

# 2. Find the most similar words to a given word
target_word = 'technology'
similar_words = wv.most_similar(target_word, topn=5)
print(f"\nTop 5 words similar to '{target_word}':")
for word, score in similar_words:
    print(f"- {word}: {score:.4f}")

# 3. Analogy task: king - man + woman = ?
result = wv.most_similar(positive=['woman', 'king'], negative=['man'], topn=1)
print(f"\nAnalogy (king - man + woman): {result[0][0]}")
```

Loading Word2Vec model (Google News 300)... This may take a minute.
 [=====] 100.0% 1662.8/1662.8MB downloaded
 Similarity between 'king' and 'queen': 0.6511

Top 5 words similar to 'technology':

- technologies: 0.8332
- innovations: 0.6231
- technological_innovations: 0.6102
- technol: 0.6047
- technological_advancement: 0.6036

Analogy (king - man + woman): queen

```
import gensim.downloader as api

# Load Word2Vec model
print("Loading Word2Vec model...")
wv = api.load('word2vec-google-news-300')

# Function to find neighbor words
def find_neighbors(target_word, n=10):
    if target_word in wv:
        print(f"Neighbors for '{target_word}':")
        neighbors = wv.most_similar(target_word, topn=n)
        for i, (word, score) in enumerate(neighbors, 1):
            print(f"{i}. {word} (Similarity: {score:.4f})")
    else:
        print(f"Word '{target_word}' not found in vocabulary.")

# Example usage
find_neighbors('space', n=5)
```

```

Loading Word2Vec model...
Neighbors for 'space':
1. spaces (Similarity: 0.6571)
2. music_concept_ShockHound (Similarity: 0.5850)
3. Shuttle_docks (Similarity: 0.5567)
4. Space (Similarity: 0.5478)
5. Soviet_Union_Yuri_Gagarin (Similarity: 0.5418)

```

```

import gensim.downloader as api

# Load the GloVe pre-trained model
print("Loading GloVe model (glove-wiki-gigaword-100)...")
glove_wv = api.load('glove-wiki-gigaword-100')

# 1. Check similarity between two words
w1, w2 = 'coffee', 'tea'
sim = glove_wv.similarity(w1, w2)
print(f"Similarity between '{w1}' and '{w2}': {sim:.4f}")

# 2. Find words most similar to 'science'
target = 'science'
similar_list = glove_wv.most_similar(target, topn=5)
print(f"\nTop 5 words similar to '{target}':")
for word, score in similar_list:
    print(f"- {word}: {score:.4f}")

# 3. Combining words (Addition)
# e.g., doctor + machine
result = glove_wv.most_similar(positive=['doctor', 'machine'], topn=1)
print(f"\n'doctor' + 'machine' most similar to: {result[0][0]}")

```

```

Loading GloVe model (glove-wiki-gigaword-100)...
Similarity between 'coffee' and 'tea': 0.7733

```

```

Top 5 words similar to 'science':
- sciences: 0.8073
- physics: 0.7915
- institute: 0.7663
- mathematics: 0.7608
- studies: 0.7590

```

```

'doctor' + 'machine' most similar to: hand

```

```

import gensim.downloader as api

# Load the GloVe pre-trained model
print("Loading GloVe model...")
glove_wv = api.load('glove-wiki-gigaword-100')

# Function to find neighbor words
def find_glove_neighbors(target_word, n=10):
    target_word = target_word.lower() # GloVe is case-sensitive and typically lowercase
    if target_word in glove_wv:
        print(f"Neighbors for '{target_word}':")
        neighbors = glove_wv.most_similar(target_word, topn=n)
        for i, (word, score) in enumerate(neighbors, 1):
            print(f"{i}. {word} (Similarity: {score:.4f})")
    else:

```

```

        print(f"Word '{target_word}' not found in GloVe vocabulary.")

# Example usage
find_glove_neighbors('planet', n=5)

```

```

Loading GloVe model...
Neighbors for 'planet':
1. earth (Similarity: 0.8551)
2. mars (Similarity: 0.7346)
3. planets (Similarity: 0.7300)
4. universe (Similarity: 0.6928)
5. pluto (Similarity: 0.6660)

```

```

import gensim.downloader as api

# Load the GloVe pre-trained model
print("Loading GloVe model...")
glove_wv = api.load('glove-wiki-gigaword-100')

# Function to perform analogy: a is to b as c is to ?
def solve_glove_analogy(a, b, c):
    # GloVe is case-sensitive and mostly lowercase
    a, b, c = a.lower(), b.lower(), c.lower()
    try:
        # Equation: result = b - a + c
        result = glove_wv.most_similar(positive=[b, c], negative=[a], topn=1)
        print(f"Analogy: '{a}' is to '{b}' as '{c}' is to '{result[0][0]}'")
    except KeyError as e:
        print(f"Error: {e}")

# Example tasks
solve_glove_analogy('man', 'king', 'woman')
solve_glove_analogy('japan', 'tokyo', 'france')
solve_glove_analogy('tall', 'taller', 'short')

```

```

Loading GloVe model...
Analogy: 'man' is to 'king' as 'woman' is to 'queen'
Analogy: 'japan' is to 'tokyo' as 'france' is to 'paris'
Analogy: 'tall' is to 'taller' as 'short' is to 'shorter'

```

```

import gensim.downloader as api

# Load Word2Vec model (this might take a moment if not already in cache)
print("Loading Word2Vec model...")
wv = api.load('word2vec-google-news-300')

# Function to perform analogy: a is to b as c is to ?
def solve_analogy(a, b, c):
    # Equation: result = b - a + c => positive=[b, c], negative=[a]
    try:
        result = wv.most_similar(positive=[b, c], negative=[a], topn=1)
        print(f"Analogy: '{a}' is to '{b}' as '{c}' is to '{result[0][0]}'")
    except KeyError as e:
        print(f"Error: {e}")

# Example: man is to king as woman is to ?
solve_analogy('man', 'king', 'woman')

```

```
# Example: paris is to france as berlin is to ?  
solve_analogy('paris', 'france', 'berlin')
```

Loading Word2Vec model...

Analogy: 'man' is to 'king' as 'woman' is to 'queen'

Analogy: 'paris' is to 'france' as 'berlin' is to 'germany'